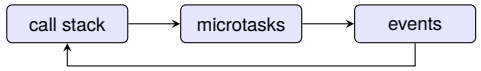
Async Dart, SQLite and offline-first apps

Worked example: a Student Task Manager (HTTP + SQLite)

Key ideas

- **HTTP** moves data between systems; a **database** keeps it. Async Dart coordinates both without freezing the UI.
- `await` suspends *one async function*, not the UI isolate.
- Marking heavy computation `async` does **not** make it non-blocking.

The event loop



Each isolate has its own loop. Microtasks run before the next event (too many starve events). Network and disk I/O complete outside Dart, then come back as events.

Future, async, await

```dart
Future<String> fetchName() async {
  await Future.delayed(
      const Duration(seconds: 2));
  return 'Amina';
}
Future<void> load() async {
  print('1. Start');
  await Future.delayed(
      const Duration(seconds: 2));
  print('2. Completed');
}
void main() { load(); print('3. Still running'); }
// prints 1, 3, 2
```

- `Future<T>`: one value of type T *or* an error, later. `Future<void>`: only "done".
- `async` makes the function return a Future at once.

```dart
getTitle().then((t) => print(t)); // callback
final t = await getTitle();       // preferred
```

Sequential or concurrent

```dart
// ~4 s: B waits for A
final a = await task('A');
final b = await task('B');
// ~2 s: both at once
final r = await Future.wait(
    [task('A'), task('B')]);
```

- Use sequential when B needs A's result.
- `Future.wait`: one failure fails the whole; the others are **not** cancelled. Default `eagerError: false` waits for all.
- Need partial results? Handle each Future on its own.

Errors and finally

```dart
try {
  final data = await fetchRemoteData();
} on TimeoutException {
  print('Request timed out');
} on FormatException {
  print('Invalid response format');
} catch (error) {
  print('Unexpected: $error');
} finally {
  print('Finished attempting load');
}
```

`finally` runs after success *and* failure: it does not mean success.

Future or Stream?

	Future	Stream
Values	one completion	zero or more events
Use	HTTP GET, SQLite insert	WebSocket, live chat, watched query
Widget	`FutureBuilder`	`StreamBuilder`

`sqflite` returns Futures, not live streams; `drift` offers watched queries.

HTTP is not a database

	Remote API	Local SQLite
Job	talk to a service	keep structured data
Where	server	device
Call	`http.get()`	`db.query()`
Offline	may fail	still works

```
flutter pub add http sqflite path
```

Files: `models/task.dart`, `data/task_api.dart`, `data/task_dao.dart`, `data/task_repository.dart`.

Never put MySQL/PostgreSQL server credentials or a Supabase service-role key in a Flutter app. Go through a protected API, with authentication and Row Level Security.

Open SQLite

```dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<Database> openTaskDb() async {
  final path = join(
      await getDatabasesPath(), 'tasks.db');
  return openDatabase(path, version: 1,
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE tasks (
          id INTEGER PRIMARY KEY,
          title TEXT NOT NULL,
          completed INTEGER NOT NULL)''');
    });
}
```

`onCreate` runs only when the file is new. A schema change needs a higher `version` and `onUpgrade`. SQLite has no `bool`: store 0/1.

CRUD

```dart
await db.insert('tasks', {
  'id': t.id, 'title': t.title,
  'completed': t.completed ? 1 : 0,
```

Copyrighted content Dr Mohamad AOUDE

```
}, conflictAlgorithm: ConflictAlgorithm.replace);

final rows = await db.query('tasks',
    orderBy: 'id');
final tasks = rows.map((r) => Task(
  id: r['id'] as int,
  title: r['title'] as String,
  completed: (r['completed'] as int) == 1,
)).toList();

await db.update('tasks', {'completed': 1},
    where: 'id = ?', whereArgs: [id]);
await db.delete('tasks',
    where: 'id = ?', whereArgs: [id]);

await db.transaction((txn) async {
  for (final t in tasks) {
    await txn.insert('tasks', t.toMap(),
      conflictAlgorithm:
          ConflictAlgorithm.replace);
  }
});
```

- `replace` = a chosen upsert policy, not synchronisation.
- Inside a transaction use `txn`, not db. All writes succeed or none.

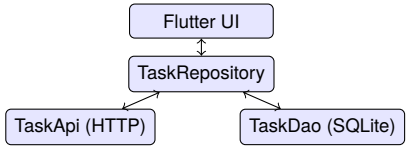
Use `where: 'id = ?', whereArgs: [x]`. Never build SQL from user input: `"... title = '$input'"` is SQL injection.

Storage libraries

`sqflite`	Future-based SQLite: the core lesson
`drift`	typed SQL, watched query Streams
`sembast`	async document / key-value store
Isar / Hive	version-specific APIs
Supabase	hosted Postgres through a client API

A `Future<List<Task>>` is a snapshot: the UI does not update when the table changes.

The repository



The UI calls repository methods; it knows nothing about HTTP or SQL. The offline policy lives in the repository.

```
// network-first, cache fallback
Future<List<Task>> load() async {
  try {
    final fresh = await api.fetchTasks();
    await db.transaction((txn) async {
      for (final t in fresh) {
        await txn.insert('tasks', t.toMap(),
          conflictAlgorithm:
              ConflictAlgorithm.replace);
      }
    });
    return fresh;
  } catch (_) {
    return readTasks(db);
  }
}
```

In production, tell network errors from database errors, and do not report an empty cache as a successful refresh.

Choose a strategy

Strategy	Gain	Cost
Network-first	fresh data	slow first screen
Cache-first, then refresh	instant display	may be stale
Write-through / outbox	offline edits	conflict resolution

Cache-first: show SQLite → refresh over HTTP → validate → save in a transaction → notify the UI; on failure keep the cache and show "Offline: showing saved data".

Race conditions

```
int requestVersion = 0;
Future<void> refresh() async {
  final version = ++requestVersion;
  final tasks = await repository.load();
  if (!mounted ||
      version != requestVersion) return;
  setState(() => visibleTasks = tasks);
}
```

An older answer cannot overwrite a newer one. It does not cancel the request.

CPU-heavy work

```
List<dynamic> parseLargeJson(String text) =>
    jsonDecode(text) as List<dynamic>;
final items =
    await compute(parseLargeJson, largeBody);
```

`await` I/O; move heavy parsing to another isolate with `compute` (`package:flutter/foundation.dart`).

Test the database and the repository

- Isolated test database; `sqflite_common_ffi` runs SQLite on the Dart VM.
- Test insert, query, update, delete, rollback, duplicate IDs; close and clean up after each test.

Scenario	Expected
API success	tasks returned and saved
No network + cache	cached tasks, offline state
No network, no cache	clear error or empty offline state
Invalid JSON	handled by the policy
Database failure	no false "saved"
Competing refreshes	old data never wins

Spot the bug

`Navigator.of(context)` used after an `await`	check `mounted` first
`repository.refresh();` alone	await it or handle its errors

Copyrighted content Dr Mohamad AOUDE